SQL Window Functions & Clauses

Practical notes for Data Science, AI/ML, SQL, Power BI, and analytics students.

1. RANK()

Definition: RANK() assigns a ranking to rows according to the specified ORDER BY. Rows with equal values receive the same rank, and the next rank is skipped.

Example:

```
SELECT Name, Salary,
RANK() OVER (ORDER BY Salary DESC) AS salary_rank
FROM Employees;
```

Example result: 90000 → 1, 80000 → 2, 80000 → 2, 70000 → 4.

2. ROW_NUMBER()

Definition: ROW_NUMBER() assigns a unique sequential number to every row. Even rows with equal values receive different row numbers.

```
SELECT Name, Salary,
ROW_NUMBER() OVER (ORDER BY Salary DESC) AS row_num
FROM Employees;
```

Example result: 90000 → 1, 80000 → 2, 80000 → 3, 70000 → 4.

3. DENSE_RANK()

Definition: DENSE_RANK() assigns the same rank to tied values, but unlike RANK(), it does not leave gaps after ties.

```
SELECT Name, Salary,
DENSE_RANK() OVER (ORDER BY Salary DESC) AS dense_rank
FROM Employees;
```

Example result: 90000 → 1, 80000 → 2, 80000 → 2, 70000 → 3.

4. PARTITION BY

Definition: PARTITION BY divides the result set into independent groups. The window function is calculated separately within each group.

```
SELECT Department, Name, Salary,
RANK() OVER (
  PARTITION BY Department
  ORDER BY Salary DESC
) AS department_rank
FROM Employees;
```

For each department, ranking starts again from 1. PARTITION BY does not reduce the number of rows.

5. ORDER BY

Definition: ORDER BY inside a window function determines the order in which rows are evaluated for ranking or other window calculations.

ORDER BY Salary DESC → highest salary gets rank 1.
ORDER BY Salary ASC → lowest salary gets rank 1.

6. ROWS

Definition: ROWS defines a window frame using physical rows relative to the current row.

```
SELECT Date, Sales,
SUM(Sales) OVER (
  ORDER BY Date
  ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
) AS rolling_sales
FROM Sales;
```

ROWS BETWEEN 2 PRECEDING AND CURRENT ROW means the calculation includes the previous two physical rows plus the current row.

7. RANGE

Definition: RANGE defines a window frame based on the values of the ORDER BY expression. Rows with equal ordering values can be treated as peers within the same range boundary.

```
SUM(Salary) OVER (
  ORDER BY Salary
  RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS cumulative_salary
```

Remember: ROWS is physical-row based, while RANGE is value/logical-range based.

8. General Window Function Syntax

```
FUNCTION() OVER (
  PARTITION BY column
  ORDER BY column
  ROWS/RANGE window_frame
)
```

9. Complete Example

```
SELECT
  Department,
  Name,
  Salary,
  RANK() OVER (
    PARTITION BY Department
    ORDER BY Salary DESC
  ) AS rank,
  ROW_NUMBER() OVER (
    PARTITION BY Department
    ORDER BY Salary DESC
  ) AS row_number,
  DENSE_RANK() OVER (
    PARTITION BY Department
    ORDER BY Salary DESC
  ) AS dense_rank
FROM Employees;
```

10. RANK vs ROW_NUMBER vs DENSE_RANK

Function	Duplicate values	Gaps after ties	Example
ROW_NUMBER()	No	No	1, 2, 3, 4
RANK()	Yes	Yes	1, 2, 2, 4
DENSE_RANK()	Yes	No	1, 2, 2, 3

11. Quick Revision

Concept	Easy Meaning
ROW_NUMBER()	Unique numbering for every row
RANK()	Same rank for ties, gaps are created
DENSE_RANK()	Same rank for ties, no gaps
PARTITION BY	Divide data into independent groups
ORDER BY	Define the ordering
ROWS	Physical-row based window frame
RANGE	Value/logical-range based window frame

Key takeaway: Window functions calculate values across related rows without collapsing the result set. They are especially useful for ranking, running totals, moving averages, top-N analysis, and time-series analytics.